NOTE: Use File > Save a copy in Drive to make a copy before doing anything else

Project 7: Student Habits vs Academic Performance Analysis

Overview

This project analyzes the relationship between student lifestyle habits and academic performance using a comprehensive dataset from Kaggle. The dataset contains information about 1,000 students and includes 16 variables covering various aspects of student life, including study habits, social media usage, sleep patterns, diet quality, exercise frequency, and academic outcomes.

The analysis focuses on understanding how different study patterns correlate with exam performance.The project demonstrates fundamental data analysis skills including data cleaning, statistical calculations, and comparative analysis using Python and pandas.

```
# Install dependencies as needed:
# pip install kagglehub[pandas-datasets]
import kagglehub
from kagglehub import KaggleDatasetAdapter

# Set the path to the file you'd like to load
# Update file_path to point to the specific file within the dataset
file_path = "student_habits_performance.csv"

# Load the latest version
df = kagglehub.load_dataset(
  KaggleDatasetAdapter.PANDAS,
  "jayaantanaath/student-habits-vs-academic-performance",
  file_path,
  # Provide any additional arguments like
  # sql_query or pandas_kwargs. See the
  # documenation for more information:
  # https://github.com/Kaggle/kagglehub/blob/main/README.md#kaggledatasetada
)
```

```
print("First 5 records:", df.head())
```

```
/tmp/ipykernel_11943/3453289196.py:11: DeprecationWarning: Use dataset_load()
  df = kagglehub.load_dataset(
Using Colab cache for faster access to the 'student-habits-vs-academic-perfor
First 5 records:   student_id  age  gender  study_hours_per_day  social_media
0      S1000   23  Female                  0.0                1.2
1      S1001   20  Female                  6.9                2.8
2      S1002   21    Male                  1.4                3.1
3      S1003   23  Female                  1.0                3.9
4      S1004   19  Female                  5.0                4.4

   netflix_hours part_time_job  attendance_percentage  sleep_hours  \
0            1.1            No                   85.0          8.0
1            2.3            No                   97.3          4.6
2            1.3            No                   94.8          8.0
3            1.0            No                   71.0          9.2
4            0.5            No                   90.9          4.9

  diet_quality  exercise_frequency parental_education_level internet_quality
0         Fair                   6                   Master          Average
1         Good                   6              High School          Average
2         Poor                   1              High School             Poor
3         Poor                   4                   Master             Good
4         Fair                   3                   Master             Good

   mental_health_rating extracurricular_participation  exam_score
0                     8                            Yes        56.2
1                     8                             No       100.0
2                     1                             No        34.3
3                     1                            Yes        26.8
4                     1                             No        66.4
```

```
one_col = df['age']
```

```
type(one_col)
```

pandas.core.series.Series
def __init__(data=None, index=None, dtype: Dtype | None=None, name=None, copy: bool | None=None, fastpath: bool | lib.NoDefault=lib.no_default) -> None

One-dimensional ndarray with axis labels (including time series).

Labels need not be unique but must be a hashable type. The object supports both integer- and label-based indexing and provides a host of methods for performing operations involving the index. Statistical methods from ndarray have been overridden to automatically exclude

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1000 entries, 0 to 999
Data columns (total 16 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   student_id                      1000 non-null   object
 1   age                             1000 non-null   int64
 2   gender                          1000 non-null   object
 3   study_hours_per_day             1000 non-null   float64
 4   social_media_hours              1000 non-null   float64
 5   netflix_hours                   1000 non-null   float64
 6   part_time_job                   1000 non-null   object
 7   attendance_percentage           1000 non-null   float64
 8   sleep_hours                     1000 non-null   float64
 9   diet_quality                    1000 non-null   object
 10  exercise_frequency              1000 non-null   int64
 11  parental_education_level        909 non-null    object
 12  internet_quality                1000 non-null   object
 13  mental_health_rating            1000 non-null   int64
 14  extracurricular_participation   1000 non-null   object
 15  exam_score                      1000 non-null   float64
dtypes: float64(6), int64(3), object(7)
memory usage: 125.1+ KB
```

```
df.shape
```

```
(1000, 16)
```

```
df.columns
```

```
Index(['student_id', 'age', 'gender', 'study_hours_per_day',
       'social_media_hours', 'netflix_hours', 'part_time_job',
       'attendance_percentage', 'sleep_hours', 'diet_quality',
       'exercise_frequency', 'parental_education_level', 'internet_quality',
       'mental_health_rating', 'extracurricular_participation',
'exam_score'],
      dtype='object')
```

clean the data

```
df.isnull().sum()
```

	0
student_id	0
age	0
gender	0
study_hours_per_day	0
social_media_hours	0

social_media_hours	0
netflix_hours	0
part_time_job	0
attendance_percentage	0
sleep_hours	0
diet_quality	0
exercise_frequency	0
parental_education_level	91
internet_quality	0
mental_health_rating	0
extracurricular_participation	0
exam_score	0

dtype: int64

This checks how many missing (NaN) values are in each column of the DataFrame. Based on the output:

All columns have 0 missing values except for parental_education_level, which has 91 missing values.

The dataset is mostly clean, but you do need to clean or handle the missing data in the parental_education_level column.

Fill with a default or common value

drop rows with missing values

↳ 2 cells hidden

Questions to Answer

Please find the answer for the following questoins.

1. Find the average study hours per day for all students. Please create a code cell below this to answer the question.(0.5 point)

```
df.describe()
#df.describe already calculates the mean, so we can just use this here - mea
```

	age	study_hours_per_day	social_media_hours	netflix_hours	at
count	909.000000	909.000000	909.000000	909.000000	
mean	20.475248	3.538724	2.504620	1.830363	
std	2.302721	1.469730	1.164802	1.071251	
min	17.000000	0.000000	0.000000	0.000000	
25%	18.000000	2.500000	1.700000	1.000000	
50%	20.000000	3.500000	2.500000	1.800000	
75%	22.000000	4.500000	3.300000	2.600000	
max	24.000000	8.300000	7.200000	5.400000	

2. Identify the student who studies MOST hours per day. Please create a code cell below to answer the question.(0.5 point)

```
df.nlargest(1, "study_hours_per_day")
#this just gets the information of the student with the most study hours - i
```

	student_id	age	gender	study_hours_per_day	social_media_hours	netfli
455	S1455	19	Male	8.3	3.3	

3. Count how many students study more than 6 hours per day. Please create a code cell below this to answer the question.(0.5 point)

```
number = df[df['study_hours_per_day'] > 6]
count_students = len(number)
#above two defines number as the count of items in the dataframe with study_ho
print(count_students)
```

```
44
```

4. What is the percentage of students who study more than 6 hours per day. Please create a code cell below this to answer the question.(0.5 point)

```
total students = len(df)
```

```
percentage = (count_students / total_students) * 100
#percentage calculation by dividing previously defined number above six (count
print(percentage)
#round up above to get 4.4%
```

```
4.3999999999999995
```

5. Calculate what percentage of students study less than 2 hours per day. Please create a code cell below this to answer the question.(0.5 point)

```
number = df[df['study_hours_per_day'] < 2]
count_studentslower = len(number)
#reuse earlier code, just make it less than 2 now
total_students = len(df)
percentage = (count_studentslower / total_students) * 100
#same as prior - counts whole list, divides this count (changed name to avoid
print(percentage)
#should be 13.3%
```

```
13.3
```

6. Do students who study more than 5 hours per day have higher exam scores on average? Please create a code cell below to answer this question. (0.5 point)

```
students_more_than_5_hours = df[df['study_hours_per_day'] > 5]
students_5_hours_or_less = df[df['study_hours_per_day'] <= 5]
#gets the full line of data for each student, grouped into two by study hour
#exam score is found in below lines after this is grouped

average_exam_score_more_than_5 = students_more_than_5_hours['exam_score'].me
average_exam_score_5_or_less = students_5_hours_or_less['exam_score'].mean()
#uses the built-in mean function to find average exam scores for both of the

print(f"Average for 5+ hours studying: {average_exam_score_more_than_5}")
print(f"Average for =<5 hours studying: {average_exam_score_5_or_less}")
#f here to display the curly-braces as the variable instead of just one homo
```

```
Average for 5+ hours studying: 90.79419354838709
Average for =<5 hours studying: 65.71408284023669
```

Double-click (or enter) to edit

7. Use "Explain code" for the code you produced for Question 6 and summarize in your own words to show that you understood the code Gemini produced. Please create a text cell below to answer this question. (0.5 point)

First, the code has two variables that it defines: 'students_more_than_5_hours' and 'students_5_hours_or_less' - for each of these it groups by either greater than 5 study hours *or* equal to/less than 5, and gets the information for each item. Then, once these are grouped, the 3rd and 4th code-lines take each of the items in these groups, gets the corresponding exam scores, and takes the mean of each group. Finally, the 5th and 6th code-lines print out the resulting average, using f and curly braces to insert it into the string.

8. The codes produced to answer the questions use "vectorization"? Please justify your answer with an example. Please create a text cell below to answer this question. (0.5 point)